

XIII. CSS réactif — Adapter le style aux différentes tailles d'écran

LE RESPONSIVE EN UNE PHRASE

Un seul code HTML/CSS qui s'adapte automatiquement à toutes les tailles d'écran (mobile, tablette, desktop) grâce aux **MEDIA QUERIES** et aux **unités fluides**.

Sommaire

- **Introduction**
 - 1. Objectifs du chapitre
 - 2. En pratique — situation professionnelle
 - 3. Quand l'utiliser, quand l'éviter
- **A. Pourquoi et comment faire du responsive**
 - 1. La diversité des écrans en 2026
 - 2. Le prérequis absolu : la meta viewport
 - 3. Les trois piliers du responsive
- **B. Les media queries**
 - 1. Syntaxe de base
 - 2. Conditions courantes
 - 3. Combinaisons
- **C. L'approche mobile-first**
 - 1. Le principe
 - 2. Pourquoi mobile-first ?
- **D. Breakpoints : lesquels choisir ?**
 - 1. Breakpoints Bootstrap (référence industrie)
 - 2. Breakpoints Tailwind
 - 3. Approche moderne : par contenu, pas par appareil
- **E. Media queries avancées**
 - 1. Mode sombre natif
 - 2. Réduction des animations
 - 3. Hover disponible ?
 - 4. Variables CSS + media queries
- **F. Tester sur différentes tailles**
 - 1. DevTools du navigateur
 - 2. Redimensionner la fenêtre
 - 3. Tests réels
 - 4. Checklist responsive de base
- **Conclusion**
 - 1. À retenir
 - 2. Pour aller plus loin

Introduction

1. Objectifs du chapitre

À l'issue de ce chapitre, vous serez capable de :

- **Expliquer** l'approche **mobile-first** et pourquoi elle est préférable.
- **Écrire** des media queries avec `min-width`, `max-width`, `orientation`, `prefers-color-scheme`.
- **Choisir** des breakpoints pertinents pour un site donné.
- **Combiner** media queries, Flexbox/Grid et unités fluides (`%`, `rem`, `vw`) pour un layout réactif.
- **Tester** la réactivité avec les DevTools du navigateur.

2. En pratique — situation professionnelle

Votre client vous envoie un message : *le site est parfait sur mon MacBook mais illisible sur mon iPhone*. Sans responsive, vous devriez créer un second site mobile. Avec les media queries, une vingtaine de lignes de CSS supplémentaires résolvent le problème sur tous les appareils.

3. Quand l'utiliser, quand l'éviter

✓ Utiliser pour	✗ Éviter quand
Site grand public (60 % du trafic vient du mobile)	Outil interne sur écran fixe connu (gain marginal)
Mobile-first (<code>min-width</code>) plutôt que desktop-first	Multiplier les breakpoints par appareil (iPhone X, S22...)
<code>prefers-reduced-motion</code> et <code>prefers-color-scheme</code> (a11y)	Tester uniquement en DevTools (vrai mobile reste requis)

A. Pourquoi et comment faire du responsive

1. La diversité des écrans en 2026

Appareil	Largeur type
Petit smartphone	320–375 px
Smartphone standard	390–430 px
Tablette portrait	768 px
Tablette paysage / petit laptop	1024 px
Laptop standard	1366–1440 px
Écran desktop	1920–3440 px

Aujourd'hui **plus de 60 %** du trafic web vient du mobile. Un site non-responsive perd ces utilisateurs.

2. Le prérequis absolu : la meta viewport

Sans cette ligne dans le `<head>`, aucune media query ne fonctionnera correctement sur mobile :

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

Voir chapitre HTML II pour le détail. **À ne jamais oublier.**

3. Les trois piliers du responsive

1. **Unités fluides** — `%`, `rem`, `vw`, `vh` plutôt que des `px` partout.
2. **Mise en page flexible** — Flexbox et Grid (chapitre VI).
3. **Media queries** — le CSS conditionnel selon la taille d'écran.

B. Les media queries

1. Syntaxe de base

```
/* Styles par défaut (mobile-first) */
body {
  font-size: 1rem;
  padding: 1rem;
}

/* Ajoutés quand la largeur est ≥ 768px */
@media (min-width: 768px) {
  body {
    font-size: 1.125rem;
    padding: 2rem;
  }
}

/* Ajoutés quand la largeur est ≥ 1200px */
@media (min-width: 1200px) {
  body {
    max-width: 1140px;
    margin: 0 auto;
  }
}
```

2. Conditions courantes

Condition	Sens
<code>(min-width: 768px)</code>	Largeur $\geq$ 768 px
<code>(max-width: 767px)</code>	Largeur $\leq$ 767 px
<code>(orientation: landscape)</code>	Écran en paysage (largeur > hauteur)
<code>(orientation: portrait)</code>	Écran en portrait
<code>(min-resolution: 2dppx)</code>	Écran HiDPI / Retina
<code>(hover: hover)</code>	Périphérique avec hover (souris, pas tactile)

3. Combinaisons

```
/* ET logique : les deux conditions doivent être vraies */
@media (min-width: 768px) and (max-width: 1023px) {
  /* tablettes seulement */
}

/* OU logique : virgule */
@media (max-width: 767px), (orientation: portrait) {
  /* mobile OU portrait */
}

/* Exclusion */
@media not (min-width: 768px) {
  /* tout sauf ≥ 768px */
}
```

C. L'approche mobile-first

1. Le principe

Écrire le CSS pour le plus petit écran d'abord, puis ajouter des styles pour les plus grands avec `min-width`.

```
/* ❌ Desktop-first – anti-pattern */
.container {
  width: 1140px;
}
@media (max-width: 1199px) {
  .container { width: 960px; }
}
@media (max-width: 991px) {
  .container { width: 720px; }
}
@media (max-width: 767px) {
  .container { width: 100%; }
}
```

```
/* ✅ Mobile-first – préféré */
.container {
  width: 100%;           /* mobile par défaut */
  padding: 0 1rem;
}
@media (min-width: 768px) {
  .container { padding: 0 2rem; }
}
@media (min-width: 1200px) {
  .container {
    max-width: 1140px;
    margin: 0 auto;
  }
}
```

2. Pourquoi mobile-first ?

- **Performance** : le CSS mobile est plus court et chargé en priorité, important pour les utilisateurs mobile qui ont souvent des connexions plus lentes.
- **Simplicité** : on ajoute des features, on n'en enlève pas (progressive enhancement).
- **Discipline** : force à réfléchir d'abord à ce qui est essentiel (mobile) avant les fioritures (desktop).
- **Standard industrie** : Bootstrap 5, Tailwind CSS et tous les frameworks modernes sont mobile-first.

♦ ``min-width`` > ``max-width``

Utiliser `min-width` est la convention mobile-first. Réserver `max-width` pour des cas très spécifiques (exceptions ciblées sur un petit écran).

D. Breakpoints : lesquels choisir ?

1. Breakpoints Bootstrap (référence industrie)

```
/* XS (mobile)      : 0 → 575px    : défaut mobile-first (pas de media query) */
/* SM (mobile large) : 576px       */
/* MD (tablette)     : 768px       */
/* LG (laptop)       : 992px       */
/* XL (desktop)      : 1200px      */
/* XXL (grand desktop) : 1400px    */

@media (min-width: 576px) { /* SM */ }
@media (min-width: 768px) { /* MD */ }
@media (min-width: 992px) { /* LG */ }
@media (min-width: 1200px) { /* XL */ }
```

2. Breakpoints Tailwind

```
/* sm : 640px */
/* md : 768px */
/* lg : 1024px */
/* xl : 1280px */
/* 2xl : 1536px */
```

3. Approche moderne : par contenu, pas par appareil

Le conseil actuel n'est pas de choisir des breakpoints sur les *tailles d'iPhone* ou de *tailles d'iPad* (il y en a des dizaines) mais **sur la lisibilité de votre contenu**.

Méthode :

1. Designer d'abord en mobile (320px).
2. Élargir progressivement la fenêtre.
3. Quand le layout devient **moche** (trop d'espace, lignes trop longues, trop petit/étiré), ajouter un breakpoint **à cet endroit précis**.
4. Répéter.

Résultat : 2-4 breakpoints **adaptés à votre design**, pas une liste générique.

E. Media queries avancées

1. Mode sombre natif

```
body {  
  background: white;  
  color: #222;  
}  
  
@media (prefers-color-scheme: dark) {  
  body {  
    background: #111;  
    color: #eee;  
  }  
}
```

→ S'adapte automatiquement au mode sombre du système d'exploitation de l'utilisateur.

2. Réduction des animations

```
.bouton {  
  transition: transform 0.3s;  
}  
  
@media (prefers-reduced-motion: reduce) {  
  .bouton {  
    transition: none;  
  }  
}
```

→ Respecte le paramètre *réduire les animations* que certains utilisateurs activent (maux de tête, vertiges).

3. Hover disponible ?

```
.carte:hover {  
  transform: scale(1.05);  
}  
  
@media (hover: none) {  
  .carte:hover {  
    transform: none;      /* pas de hover sur tactile, évite un "stuck" */  
  }  
}
```

4. Variables CSS + media queries

Combinaison puissante :

```
:root {  
  --spacing: 1rem;  
  --max-width: 100%;  
}  
  
@media (min-width: 768px) {  
  :root {  
    --spacing: 1.5rem;  
    --max-width: 720px;  
  }  
}  
  
@media (min-width: 1200px) {  
  :root {  
    --spacing: 2rem;  
    --max-width: 1140px;  
  }  
}  
  
.section {  
  padding: var(--spacing);  
  max-width: var(--max-width);  
  margin: 0 auto;  
}
```

→ Change **toutes** les valeurs d'un coup, sans dupliquer les règles.

F. Tester sur différentes tailles

1. DevTools du navigateur

Chrome / Firefox / Edge : **F12** → icône mobile (ou **Ctrl+Shift+M**) → **Device Toolbar**.

Permet de :

- Simuler un appareil précis (iPhone 15, Galaxy S22, iPad Pro...).
- Tester une largeur arbitraire en pixels.
- Simuler une connexion lente (**Slow 3G**).
- Prendre des captures d'écran d'une page complète.

2. Redimensionner la fenêtre

Méthode brute mais efficace : diminuer progressivement la largeur de la fenêtre en la redimensionnant et observer les cassures.

3. Tests réels

Les DevTools simulent mais **ne remplacent pas un test sur un vrai téléphone**. Pour un projet pro :

- Tester sur **au moins un iPhone et un Android** réel.
- Vérifier les comportements tactiles (tap, swipe).
- Vérifier les performances (animations fluides, temps de chargement).

4. Checklist responsive de base

- ✗ `<meta viewport>` présent dans `<head>` .
- ✗ Approche mobile-first (`min-width` partout).
- ✗ Pas de débordement horizontal sur 320px (le plus petit écran usuel).
- ✗ Images fluides (`max-width: 100%; height: auto;`).
- ✗ Texte lisible sans zoom (font-size $\geq$ 16px sur mobile).
- ✗ Boutons et liens cliquables avec un doigt (zone tactile $\geq$ 44×44 px).
- ✗ Pas de hover critique sur du contenu (le tactile n'a pas de hover).
- ✗ Les formulaires utilisent les bons `type` HTML5 pour le clavier mobile.

Conclusion

1. À retenir

- **Meta viewport** dans `<head>` = prérequis absolu pour que le responsive fonctionne.
- **Mobile-first** : styles par défaut pour mobile, `min-width` pour élargir.
- **3 piliers** du responsive : unités fluides + Flexbox/Grid + media queries.
- Breakpoints courants : 576, 768, 992, 1200 px (Bootstrap) ou à définir selon le design.
- `prefers-color-scheme: dark` et `prefers-reduced-motion` pour l'accessibilité.
- Variables CSS + media queries = changements de layout propres et centralisés.
- Toujours tester en **DevTools + vrai appareil**.

2. Pour aller plus loin

- **MDN — Media queries**
- **MDN — Responsive design (guide)**
- **web.dev — Responsive Web Design Basics**
- **Bootstrap — Breakpoints**